

Answer Key — JavaScript Callbacks

Teacher-facing document. Contains model answers, worked solutions, and grading notes.

Exercise 1 — Write Your First Callback

Model solution:

```
function playSong(title, onFinished) {  
  console.log("Playing: " + title);  
  onFinished(title);  
}  
  
playSong("Blue (Da Ba Dee)", function (title) {  
  console.log("Finished playing: " + title);  
});
```

For tests to pass, students must also add at the bottom of their file:

```
module.exports = { playSong };
```

What to look for: - The callback is *called* with `onFinished(title)` — not just referenced - The callback receives the title as an argument (required for the tests) - `console.log("Playing: ...")` appears before the callback call

Common mistake: Calling `onFinished()` without passing `title`, or writing `onFinished = title` (assignment instead of call).

Grading note: - Weak: function runs without errors but callback receives no argument - Adequate: callback is called with `title`; tests pass - Strong: student adds a comment explaining why `onFinished` is passed as an argument rather than called directly (i.e., demonstrates understanding of deferred execution)

Exercise 2 — Predict the Output

Correct output:

```
Opening app...  
Waiting for your playlist...  
Playlist loaded
```

Why: `setTimeout` hands the callback to the Web API. Even with a `0ms` delay, the callback enters the callback queue and can only move to the call stack once the current synchronous code has fully finished. “Waiting for your playlist...” runs synchronously before the event loop gets a chance to process the queue.

What to look for in student comments: - They predicted `Playlist` loaded last (or not) - Their explanation references the call stack being “busy” or “not empty”

Grading note: - Weak: student just reports the output without explaining why - Adequate: student explains that `setTimeout` is non-blocking and runs after synchronous code - Strong: student uses the terms “call stack”, “callback queue”, or “event loop” correctly in their explanation

Exercise 3 — Fix the Sequence

Model solution:

```
function playTrack(title, delay, onDone) {
  setTimeout(function () {
    console.log("Now playing: " + title);
    onDone();
  }, delay);
}

playTrack("Bohemian Rhapsody", 1000, function () {
  playTrack("Blue (Da Ba Dee)", 800, function () {
    playTrack("Sandstorm", 600, function () {
      console.log("DJ set complete.");
    });
  });
});
```

Expected output:

```
Now playing: Bohemian Rhapsody    (after 1s)
Now playing: Blue (Da Ba Dee)    (after another 0.8s)
Now playing: Sandstorm           (after another 0.6s)
DJ set complete.
```

What to look for: - Each subsequent `playTrack` call is *inside* the `onDone` callback of the previous one - The empty `function() {}` callbacks in the starter code are replaced with the next call - Order is guaranteed by nesting, not by adjusting the delay values

Common mistake: Setting all delays to different values to “sort” the order, without using callbacks. This is a timing hack, not a solution — and it will break if delays change.

Grading note: - Weak: relies on delay values to control order (not using callbacks correctly) - Adequate: correctly chains all three calls; order is guaranteed - Strong: adds a final callback or log after Track 3; can explain *why* the nesting guarantees order

Exercise 4 — Error-First Callback

Model solution:

```
const available = {
  "chill": ["Weightless – Marconi Union", "Breathe – Pink Floyd"],
  "workout": ["Eye of the Tiger – Survivor", "Lose Yourself –
    Eminem"],
};

function loadPlaylist(name, callback) {
  if (available[name]) {
    callback(null, available[name]);
  } else {
    callback(new Error("Playlist not found"), null);
  }
}

loadPlaylist("chill", function (err, songs) {
  if (err) {
    console.error("Error:", err.message);
    return;
  }
  console.log("Loaded playlist:", songs);
});

loadPlaylist("techno", function (err, songs) {
  if (err) {
    console.error("Error:", err.message);
    return;
  }
  console.log("Loaded playlist:", songs);
});

module.exports = { loadPlaylist };
```

For tests to pass, students must export `loadPlaylist` and call `callback(null, songs)` / `callback(new Error(...), null)` — in that order.

What to look for: - `null` as the first argument on success (not omitted) - An actual `Error` object on failure, not just a string - The `return` after handling the error (prevents execution continuing)

Common mistake: Passing the error as the second argument, or using `callback(songs)` without the leading `null`.

Grading note: - Weak: function works for success but doesn't handle the error case
- Adequate: both cases handled; error is an Error object; tests pass - Strong: student adds a return after the error branch and can explain why it matters

Exercise 5 — Escape the Pyramid

Model solution:

```
function playTrack5(onDone) {
  setTimeout(function () {
    console.log("Track 5: Da Funk – Daft Punk");
    console.log("Set complete.");
    onDone();
  }, 500);
}

function playTrack4(onDone) {
  setTimeout(function () {
    console.log("Track 4: Harder Better Faster – Daft Punk");
    playTrack5(onDone);
  }, 500);
}

function playTrack3(onDone) {
  setTimeout(function () {
    console.log("Track 3: Get Lucky – Daft Punk");
    playTrack4(onDone);
  }, 500);
}

function playTrack2(onDone) {
  setTimeout(function () {
    console.log("Track 2: Around the World – Daft Punk");
    playTrack3(onDone);
  }, 500);
}

function playTrack1(onDone) {
  setTimeout(function () {
    console.log("Track 1: One More Time – Daft Punk");
    playTrack2(onDone);
  }, 500);
}

playTrack1(function () {});
```

Alternative (array-based) solution — acceptable for stronger students:

```
const tracks = [
  "Track 1: One More Time – Daft Punk",
  "Track 2: Around the World – Daft Punk",
```

```

    "Track 3: Get Lucky – Daft Punk",
    "Track 4: Harder Better Faster – Daft Punk",
    "Track 5: Da Funk – Daft Punk",
  ];

  function playNext(index) {
    if (index >= tracks.length) {
      console.log("Set complete.");
      return;
    }
    setTimeout(function () {
      console.log(tracks[index]);
      playNext(index + 1);
    }, 500);
  }

  playNext(0);

```

What to look for: - No nesting beyond one level deep - Output is identical to the original (same order, same text) - No Promises, async/await, or npm packages used

Grading note: - Weak: code still has 2+ levels of nesting, or output order changed - Adequate: fully flat using named functions; output matches original - Strong: uses an array/loop-based approach (discovers a reusable pattern without being prompted)

Exercise 6 — Reflection (Open Question)

There is no single correct answer. Key elements of a strong response:

- Identifies a *specific* moment (e.g., pressing play, searching for a song, buffering)
- Connects it to the idea that the UI stays responsive *while* something loads in the background
- Mentions the idea of “do this, and when it’s done, call me back” — even informally
- Shows awareness that the event doesn’t block the rest of the app

Examples of strong answers: - “When I press play on Spotify, the app doesn’t freeze — it sends a request to load the audio, and when it arrives it starts the playback. The callback is like the on load event telling the player to start.” - “YouTube shows the video thumbnail immediately while the video loads in the background. There must be a callback that fires when enough is buffered to start playing.”

Grading note: - Weak: vague (“it loads the data”) with no connection to async concepts - Adequate: identifies a specific moment and explains the non-blocking aspect - Strong: uses callback/event-loop vocabulary naturally and correctly in context